M2 PHAN PHEARAMONY
Kamal's Laboratory
Gunma University



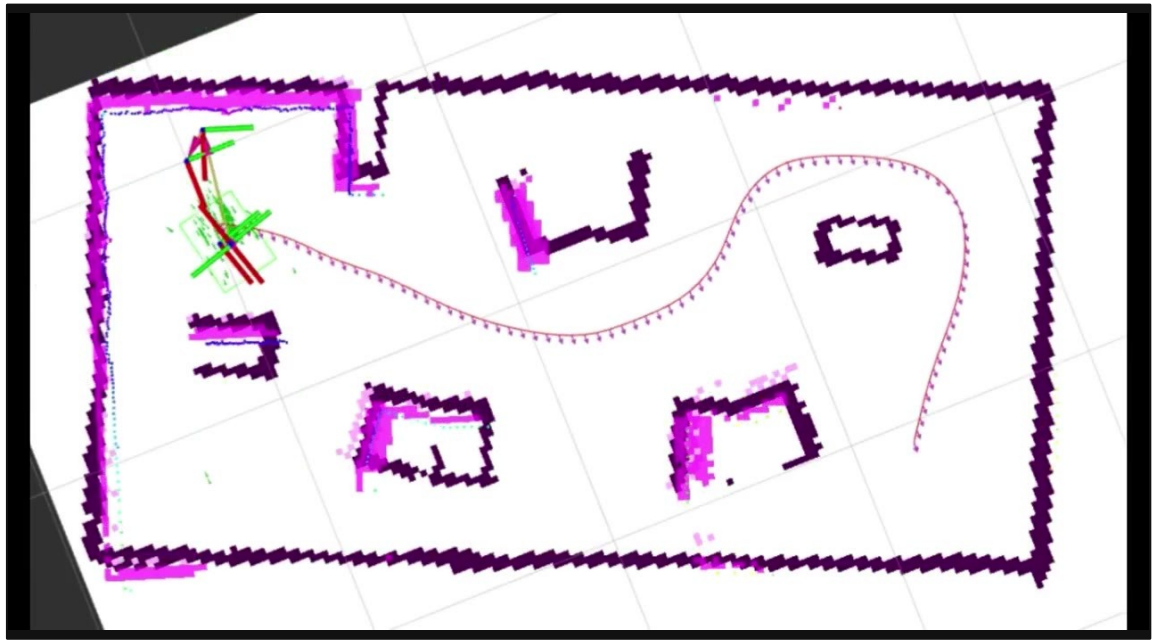
ROSMASTER CRASH COURSE

Week 1: ROS2 Introduction

GOAL: HOW A ROBOT MOVES LIKE THIS



Real robot



RViz2 (what the robot sees)

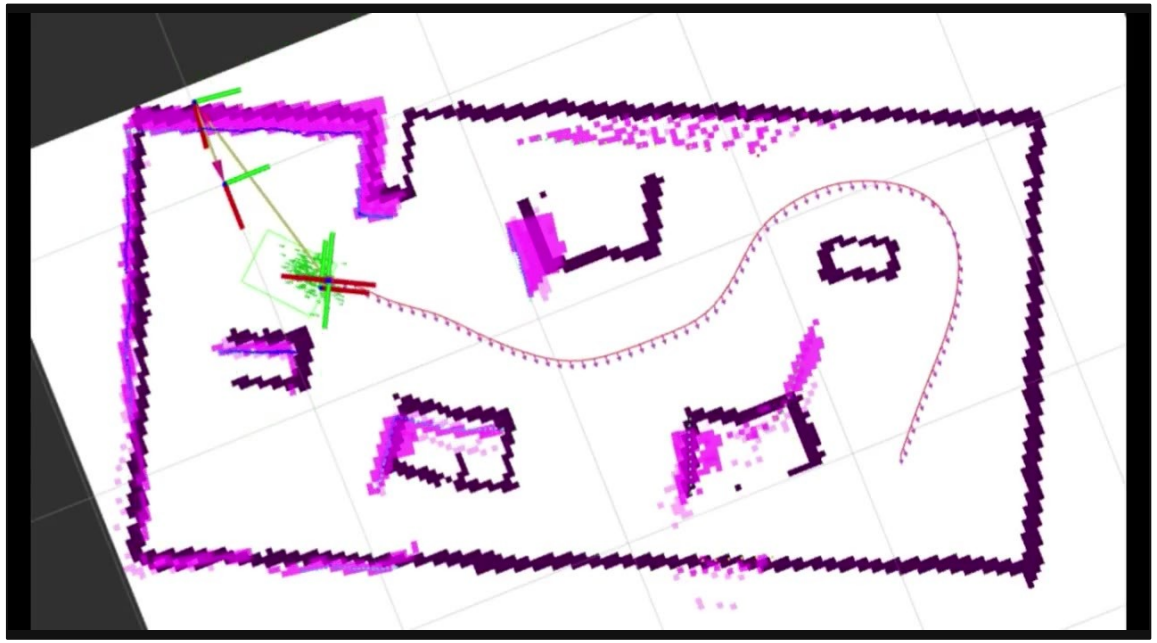
Known map — New obstacles — No map

The robot already has the map of the room. It plans a path to the goal and drives there.

GOAL: HOW A ROBOT MOVES LIKE THIS



Real robot



RViz2 (what the robot sees)

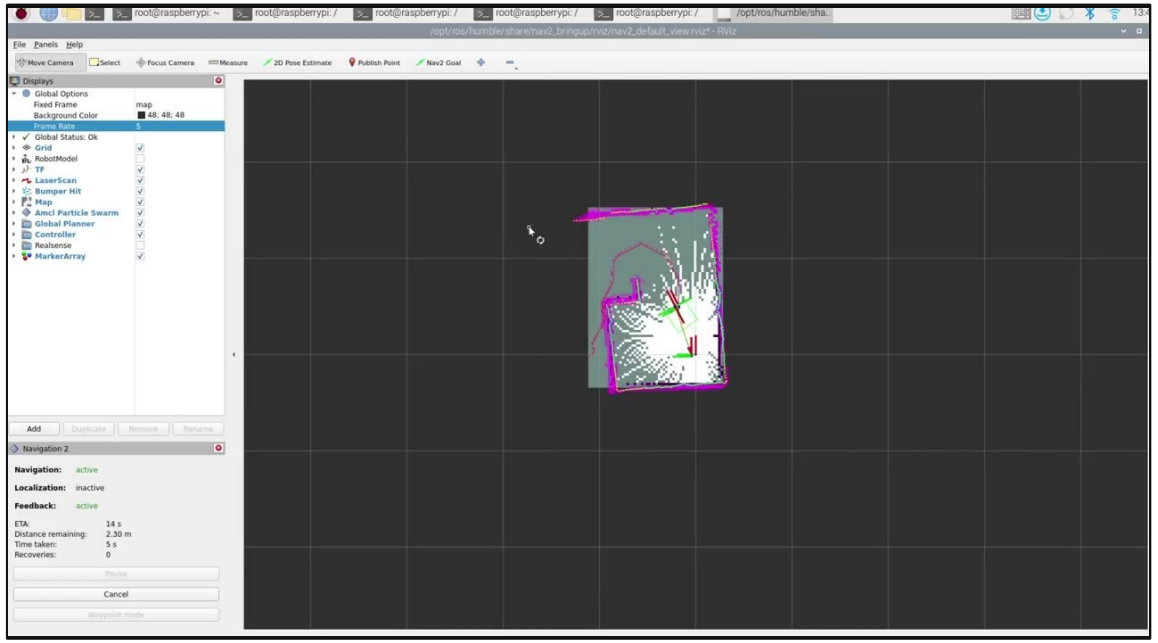
Known map — **New obstacles** — No map

Same map, but we put boxes that are not on the map. The LiDAR sees them and the robot goes around.

GOAL: HOW A ROBOT MOVES LIKE THIS



Real robot



RViz2 (what the robot sees)

Known map — New obstacles — **No map**

A new room with no map. The robot builds the map while it drives and still reaches the goal.

COURSE PLAN

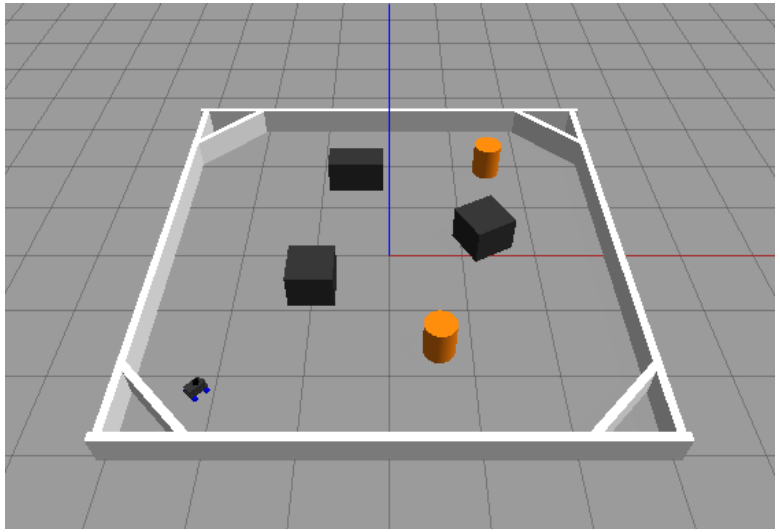
Week 1
ROS2 Introduction

Week 2
Reading the Code

Week 3
Control and Sensors

Week 4
ROSMASTER Functions

- What ROS2 is
- Nodes, topics and messages
- ROS vs ROS2, versions
- Gazebo simulator
- Hands-on: simulated A1 avoids obstacles with LiDAR



COURSE PLAN



- Linux and the terminal
- Bash: cd, ls, nano, source, sudo apt
- Python: variables, if, functions, classes
- Go through the week 1 code line by line

```
def scan_callback(self, scan):
    front = self.sector_min(...)
    if front < safe:
        turn()
```

COURSE PLAN

Week 1
ROS2 Introduction

Week 2
Reading the Code

Week 3
Control and Sensors

Week 4
ROSMASTER Functions

- How the A1 moves: /cmd_vel, Ackermann steering, odometry
- Sensors: LiDAR, camera, IMU, wheel encoders
- LiDAR obstacle avoidance on the real A1
- Homework: recognize objects with the A1's built-in YOLO



COURSE PLAN

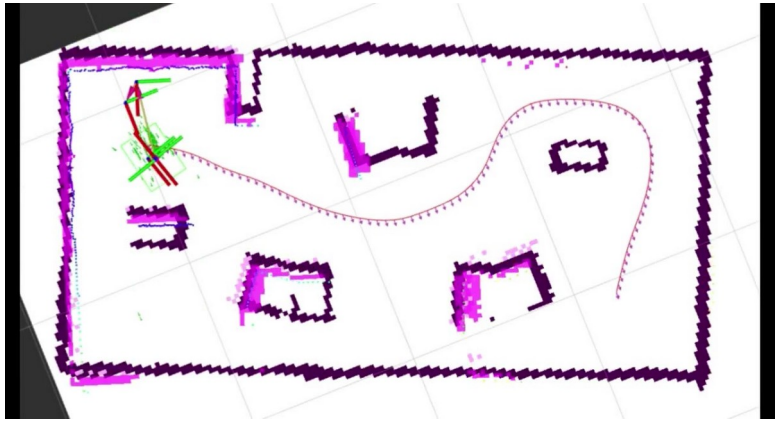
Week 1
ROS2 Introduction

Week 2
Reading the Code

Week 3
Control and Sensors

Week 4
ROSMASTER Functions

- Built-in functions from the Yahboom documentation
- SLAM mapping and Nav2 navigation
- LiDAR follow and patrol, line following, color tracking
- AI: object recognition, text or voice commands



WHAT IS ROS2?

ROS: Robot Operating System
Not really an operating system. A framework, tools and libraries for building robot software, running on Linux.



Sensors



Actuators



Computer

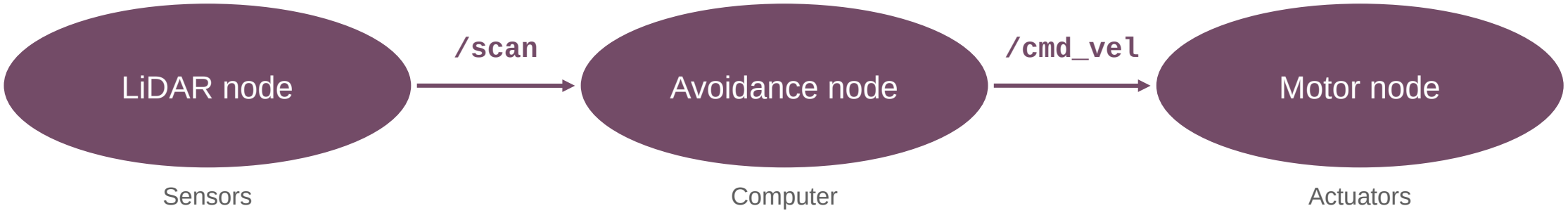
NODES

Each part of the robot is a separate program, called a node. Nodes send data to each other through topics.









Change one node without touching the others: a new LiDAR only changes the LiDAR node.

HOW NODES COMMUNICATE?

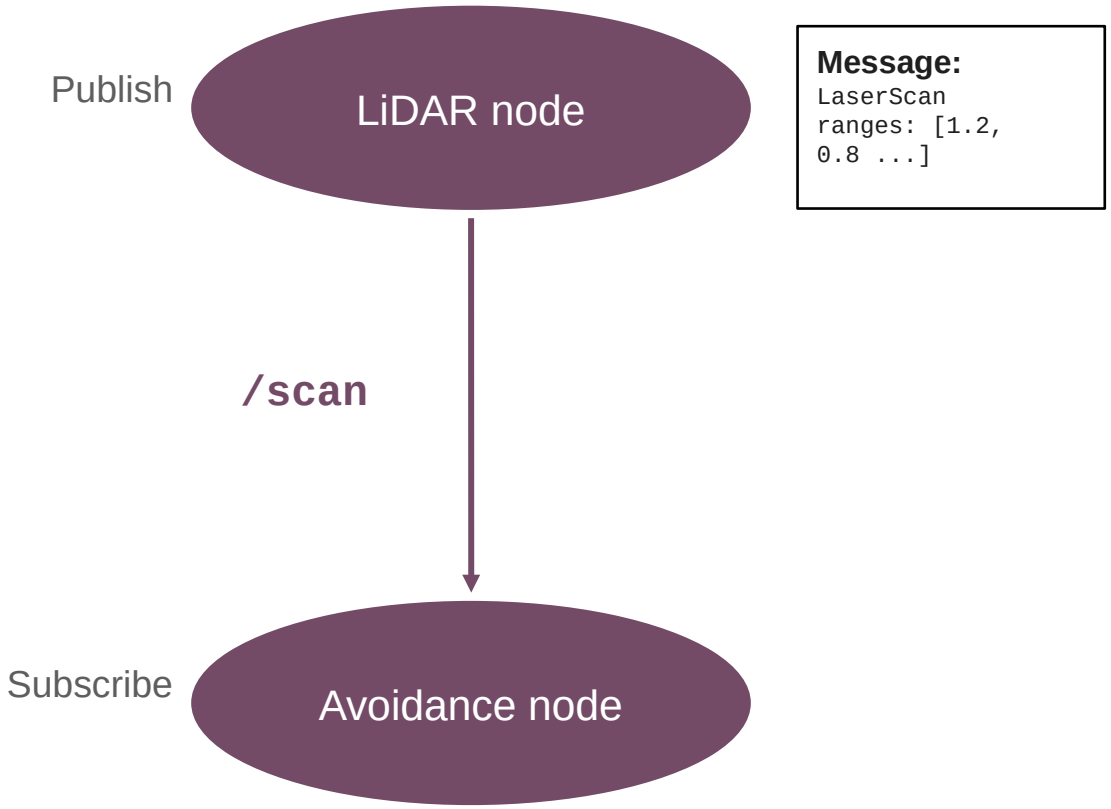
Node: a program that does one job

Topic: a named channel, e.g. /scan

Message: the data sent on a topic

Publisher: a node that sends to a topic

Subscriber: a node that listens to a topic



HOW NODES COMMUNICATE?

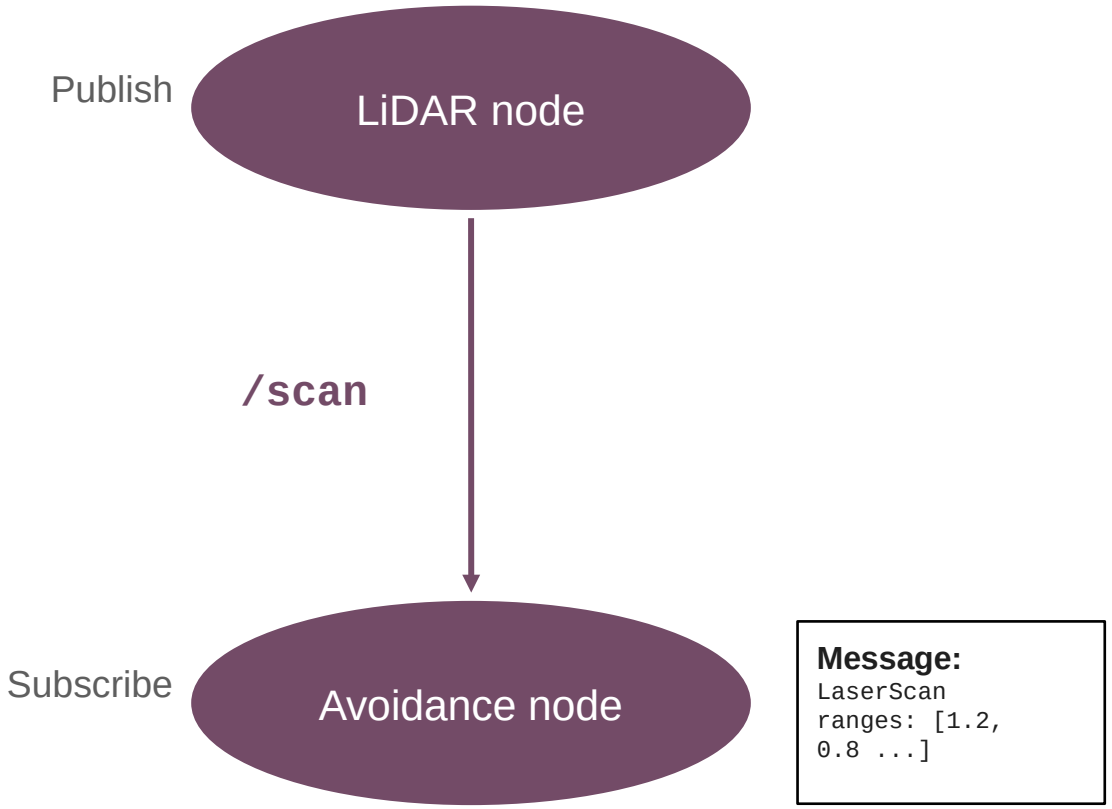
Node: a program that does one job

Topic: a named channel, e.g. /scan

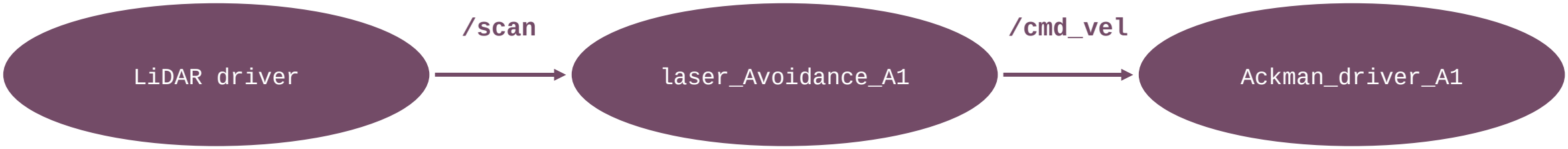
Message: the data sent on a topic

Publisher: a node that sends to a topic

Subscriber: a node that listens to a topic



ROS2 ON THE ROSMASTER A1





```
# chassis (motors + steering servo)
ros2 run yahboomcar_bringup Ackman_driver_A1
# LiDAR (C1)
ros2 launch sllidar_ros2 sllidar_c1_launch.py
# obstacle avoidance
ros2 run yahboomcar_laser laser_Avoidance_A1
```

TOPICS ON THE A1

Topic	Message type	Description	Dir
`/scan`	`sensor_msgs/LaserScan`	LiDAR distances	PUB
`/cmd_vel`	`geometry_msgs/Twist`	Speed command to the robot	SUB
`/odom`	`nav_msgs/Odometry`	Robot position estimate	PUB
`/imu`	`sensor_msgs/Imu`	Rotation and acceleration	PUB
`/map`	`nav_msgs/OccupancyGrid`	2D map	PUB
`/camera/image_raw`	`sensor_msgs/Image`	Camera image	PUB

Today: read /scan, write /cmd_vel.

ROS VS ROS2

ROS (ROS1)

- Started in 2007
- Needs a master: roscore. If it stops, everything stops
- Last version: Noetic, end of life May 31, 2025
- Tutorials say: roscore, rospy, catkin_make

ROS2

- First release December 2017
- No master. Nodes find each other on the network
- Works better with Wi-Fi and many robots
- Tutorials say: rclpy, colcon. The A1 uses this

ROS2 VERSIONS



Newest: Lyrical Luth

- Released May 22, 2026, supported until May 2031
- New version every year on May 23
- Even years are LTS: 5 years of support

ROS2 VERSIONS



We use: Humble Hawksbill

- Ubuntu 22.04, supported until May 2027
- The ROSMASTER A1 comes with Humble
- Robot, laptop and lab must use the same version

ROS2 STRUCTURE: WORKSPACE

rosmaster_ws/

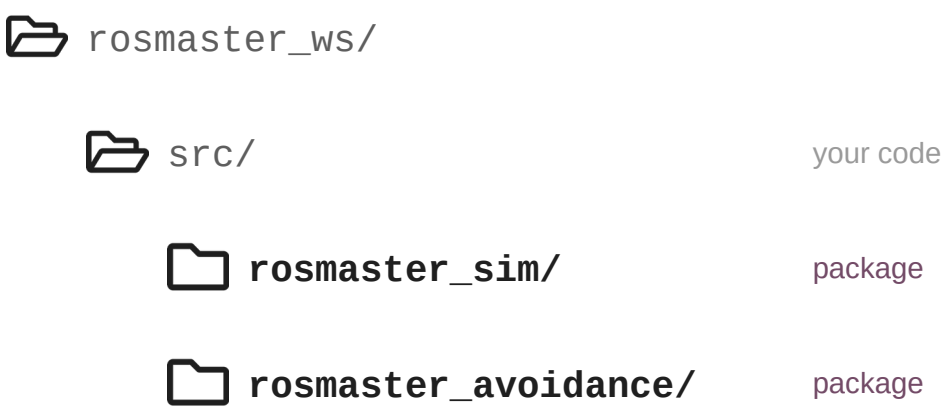
src/ — your code

A workspace is a folder that holds everything for one project.

Your code goes in src/ (source). You can name the workspace anything.

```
mkdir -p ~/rosmaster_ws/src
```

ROS2 STRUCTURE: PACKAGES

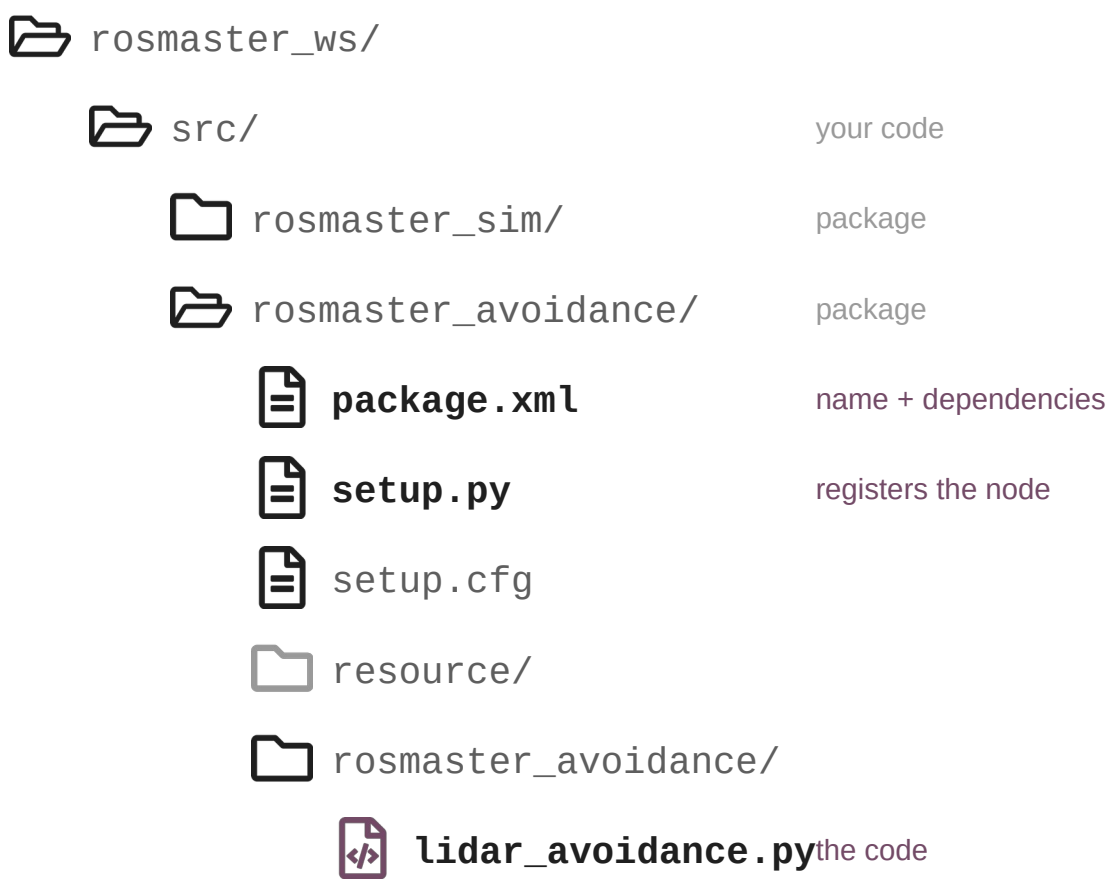


Inside src/ are packages. A package is a folder for one job. ROS2 builds and runs code by package.

`rosmaster_sim`
A1 model, Gazebo world, launch file

`rosmaster_avoidance`
our node: reads /scan, sends /cmd_vel

ROS2 STRUCTURE: INSIDE A PACKAGE



`package.xml`
name, version, and what it needs (rclpy, sensor_msgs ...)

`setup.py`
tells ROS which nodes are in the package, so ros2 run can find them

`lidar_avoidance.py`
the actual code, we read it in week 2

One command makes a new package:

```
ros2 pkg create --build-type ament_python <name>
```

ROS2 STRUCTURE: BUILD

- `rosmaster_ws/`
  - `src/` — your code
    - `rosmaster_sim/` — package
    - `rosmaster_avoidance/` — package
  - **`build/`** — made by colcon
  - **`install/`** — made by colcon
  - **`log/`** — made by colcon

colcon build reads every package in src/ and makes build/, install/ and log/. Don't edit these.

```
cd ~/rosmaster_ws
colcon build
source install/setup.bash
ros2 run <package> <node>
```

Every new terminal needs source, or ROS says "package not found".

BASIC COMMANDS

```
ros2 run <package> <node>          # start one node
ros2 launch <package> <file>       # start many nodes

ros2 node list                     # running nodes
ros2 topic list                    # topics
ros2 topic echo /scan              # print messages
ros2 topic hz /scan                # how fast

rqt_graph                          # draw the graph
```

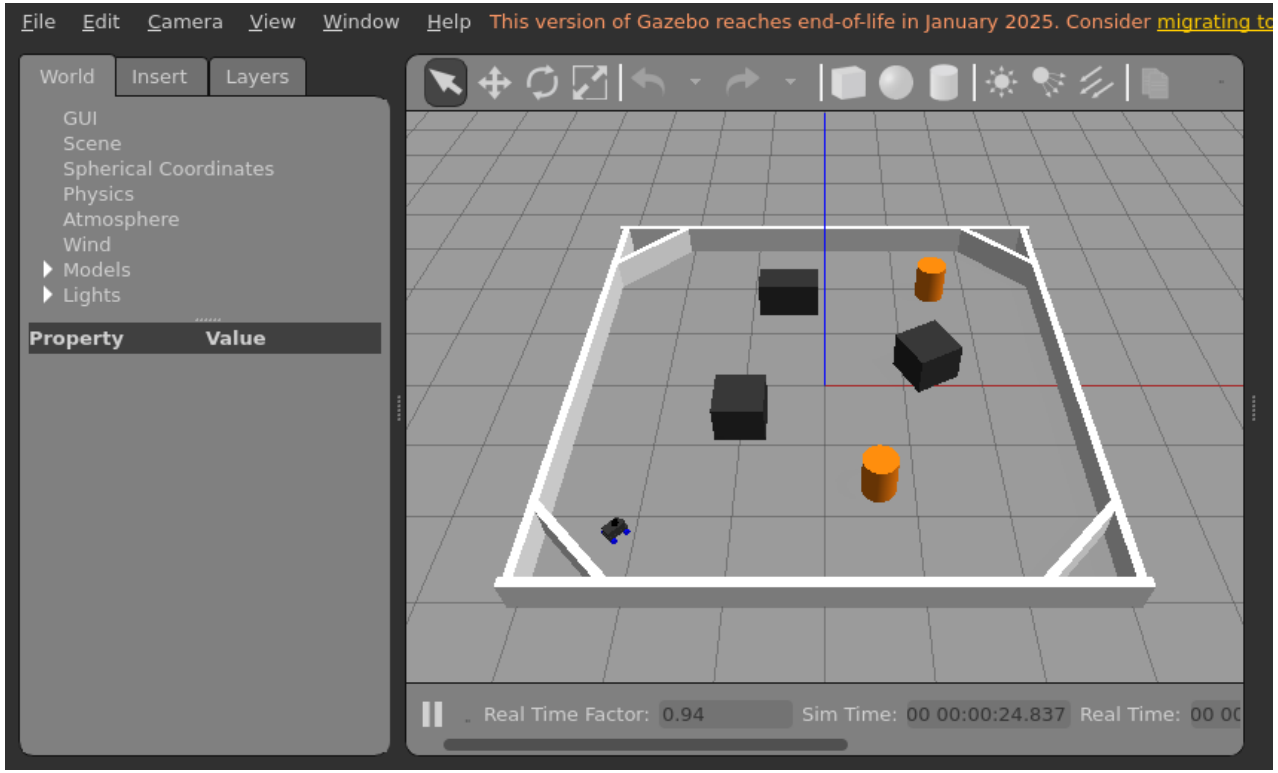
Ctrl+C stops a running node.

INTRODUCING GAZEBO

Gazebo: 3D robot simulator that works with ROS.

- Virtual world with real physics
- Virtual sensors: LiDAR, camera, IMU
- Same topics as the real robot: publishes /scan, listens to /cmd_vel
- Hit a wall 100 times, nothing breaks

We use Gazebo Classic 11 (comes with Humble, end of life January 2025).

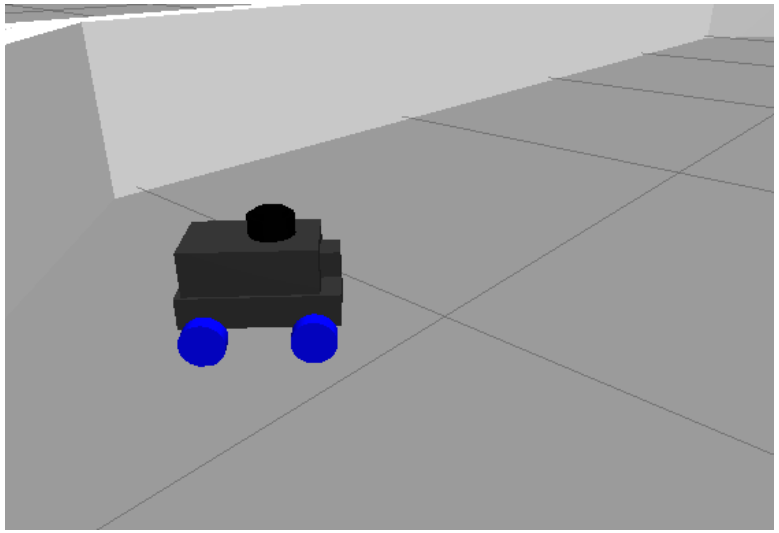


Our room: 6 x 6 m, A1 in the corner

SIMULATION VS REAL ROBOT

Gazebo

Week 1, 2



`lidar_avoidance`

`/scan in, /cmd_vel out`

Same code, no changes

Real ROSMASTER A1

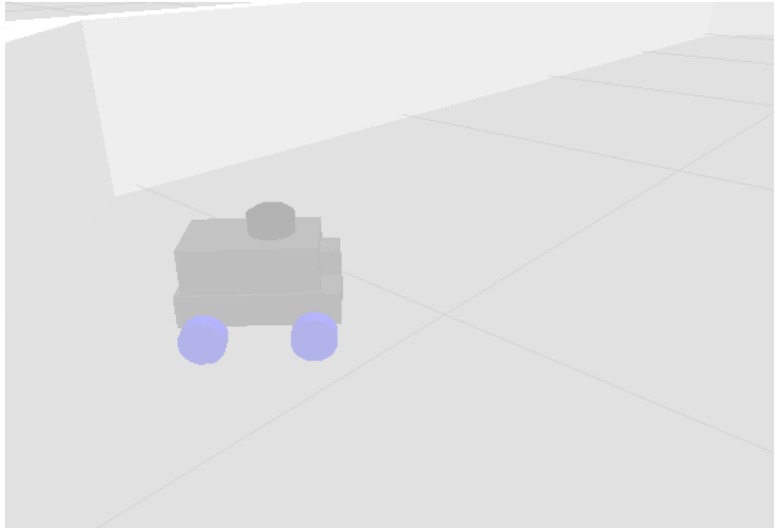
Week 3, 4



SIMULATION VS REAL ROBOT

Gazebo

Week 1, 2



`lidar_avoidance`

`/scan in, /cmd_vel out`

Same code, no changes

Real ROSMASTER A1

Week 3, 4



GETTING STARTED: SETTING UP

1. **Install Ubuntu** — 2. Prepare Ubuntu — 3. Add ROS2 source — 4. Install ROS2 + Gazebo — 5. Load ROS2 + test — 6. Test Gazebo



Windows PowerShell (Run as administrator)

```
wsl --install -d Ubuntu-22.04

# restart the PC, open "Ubuntu 22.04" from the Start menu
# make a username and password (password is invisible when typing)

lsb_release -a          # check: Ubuntu 22.04
```

WSL2 runs Ubuntu 22.04 inside Windows. ROS2 Humble needs Ubuntu 22.04.

Note: Windows 11, or Windows 10 build 19044 or newer (Win+R, winver). Already have WSL: run wsl --update first.

GETTING STARTED: SETTING UP

1. Install Ubuntu
2. **Prepare Ubuntu**
3. Add ROS2 source
4. Install ROS2 + Gazebo
5. Load ROS2 + test
6. Test Gazebo



Ubuntu terminal

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y locales software-properties-common curl git

sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
export LANG=en_US.UTF-8

sudo add-apt-repository -y universe
```

Update Ubuntu, install a few tools, set the language to UTF-8, turn on the universe packages.

Note: sudo asks for your Ubuntu password. Type it and press Enter, even if nothing shows.

GETTING STARTED: SETTING UP

1. Install Ubuntu
2. Prepare Ubuntu
3. **Add ROS2 source**
4. Install ROS2 + Gazebo
5. Load ROS2 + test
6. Test Gazebo



Ubuntu terminal

```
export ROS_APT_SOURCE_VERSION=$(curl -s \
  https://api.github.com/repos/ros-infrastructure/ros-apt-source/releases/latest \
  | grep -F "tag_name" | awk -F'"' '{print $4}')
V=$ROS_APT_SOURCE_VERSION

curl -L -o /tmp/ros2-apt-source.deb \
  https://github.com/ros-infrastructure/ros-apt-source/releases/download/$V/ros2-apt-source_$V.jammy_all.deb

sudo dpkg -i /tmp/ros2-apt-source.deb
```

apt doesn't know where ROS2 is yet. This installs a small package that adds the official ROS2 source.

Note: Copy it exactly, or copy from the README. Same as the official ROS2 Humble install guide.

GETTING STARTED: SETTING UP





Ubuntu terminal

```
sudo apt update && sudo apt upgrade -y

# ROS2 Humble + RViz + demos, and build tools (colcon)
sudo apt install -y ros-humble-desktop ros-dev-tools

# Gazebo + ROS plugins, xacro, keyboard driving
sudo apt install -y ros-humble-gazebo-ros-pkgs ros-humble-xacro \
  ros-humble-teleop-twist-keyboard
```

Installs everything for the course: ROS2 Humble, colcon, Gazebo 11 with its ROS plugins, xacro, keyboard teleop.

Note: A few GB, 20 to 40 minutes. Do this at home before class.

GETTING STARTED: SETTING UP



ROS **Ubuntu terminal**

```
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
source ~/.bashrc

ros2 run demo_nodes_cpp talker      # terminal 1
ros2 run demo_nodes_py listener     # terminal 2
```

~/.bashrc runs every time a terminal opens, so ROS2 is always loaded.

Note: The listener should print I heard: [Hello World: 1]. Stop both with Ctrl+C.

GETTING STARTED: SETTING UP



GAZEBO **Ubuntu terminal**

```
gazebo
# an empty world opens on your Windows desktop, then close it

# black window or crash? use software graphics:
echo "export LIBGL_ALWAYS_SOFTWARE=1" >> ~/.bashrc
source ~/.bashrc
```

WSL2 shows Linux windows on the Windows desktop, so Gazebo opens like a normal app.

Note: First start is slow. Updating the Windows graphics driver helps.

RUNNING THE SIMULATION

```
# 1. Get the code
mkdir -p ~/rosmaster_ws/src && cd ~/rosmaster_ws/src
git clone https://github.com/Phearamony/ROSMASTER_crash_course.git


# 2. Build
cd ~/rosmaster_ws && colcon build --symlink-install
source install/setup.bash


# 3. Terminal 1: Gazebo with the A1
ros2 launch rosmaster_sim sim.launch.py


# 4. Terminal 2 (source first): start the node
ros2 run rosmaster_avoidance lidar_avoidance
```

Step 1: Get the code

mkdir makes the workspace and src/. git clone downloads the two course packages into src/.

RUNNING THE SIMULATION

```
# 1. Get the code
mkdir -p ~/rosmaster_ws/src && cd ~/rosmaster_ws/src
git clone https://github.com/Phearamony/ROSMASTER_crash_course.git

# 2. Build
cd ~/rosmaster_ws && colcon build --symlink-install
source install/setup.bash

# 3. Terminal 1: Gazebo with the A1
ros2 launch rosmaster_sim sim.launch.py

# 4. Terminal 2 (source first): start the node
ros2 run rosmaster_avoidance lidar_avoidance
```

Step 2: Build

colcon build makes the programs in install/. --symlink-install means you can edit the Python file without building again. source tells the terminal where they are.

RUNNING THE SIMULATION

```
# 1. Get the code
mkdir -p ~/rosmaster_ws/src && cd ~/rosmaster_ws/src
git clone https://github.com/Phearamony/ROSMASTER_crash_course.git

# 2. Build
cd ~/rosmaster_ws && colcon build --symlink-install
source install/setup.bash

# 3. Terminal 1: Gazebo with the A1
ros2 launch rosmaster_sim sim.launch.py

# 4. Terminal 2 (source first): start the node
ros2 run rosmaster_avoidance lidar_avoidance
```

Step 3: Open Gazebo

The launch file starts Gazebo with the room, publishes the robot model, and puts the A1 in the corner. The robot waits for /cmd_vel.

RUNNING THE SIMULATION

```
# 1. Get the code
mkdir -p ~/rosmaster_ws/src && cd ~/rosmaster_ws/src
git clone https://github.com/Phearamony/ROSMASTER_crash_course.git

# 2. Build
cd ~/rosmaster_ws && colcon build --symlink-install
source install/setup.bash

# 3. Terminal 1: Gazebo with the A1
ros2 launch rosmaster_sim sim.launch.py

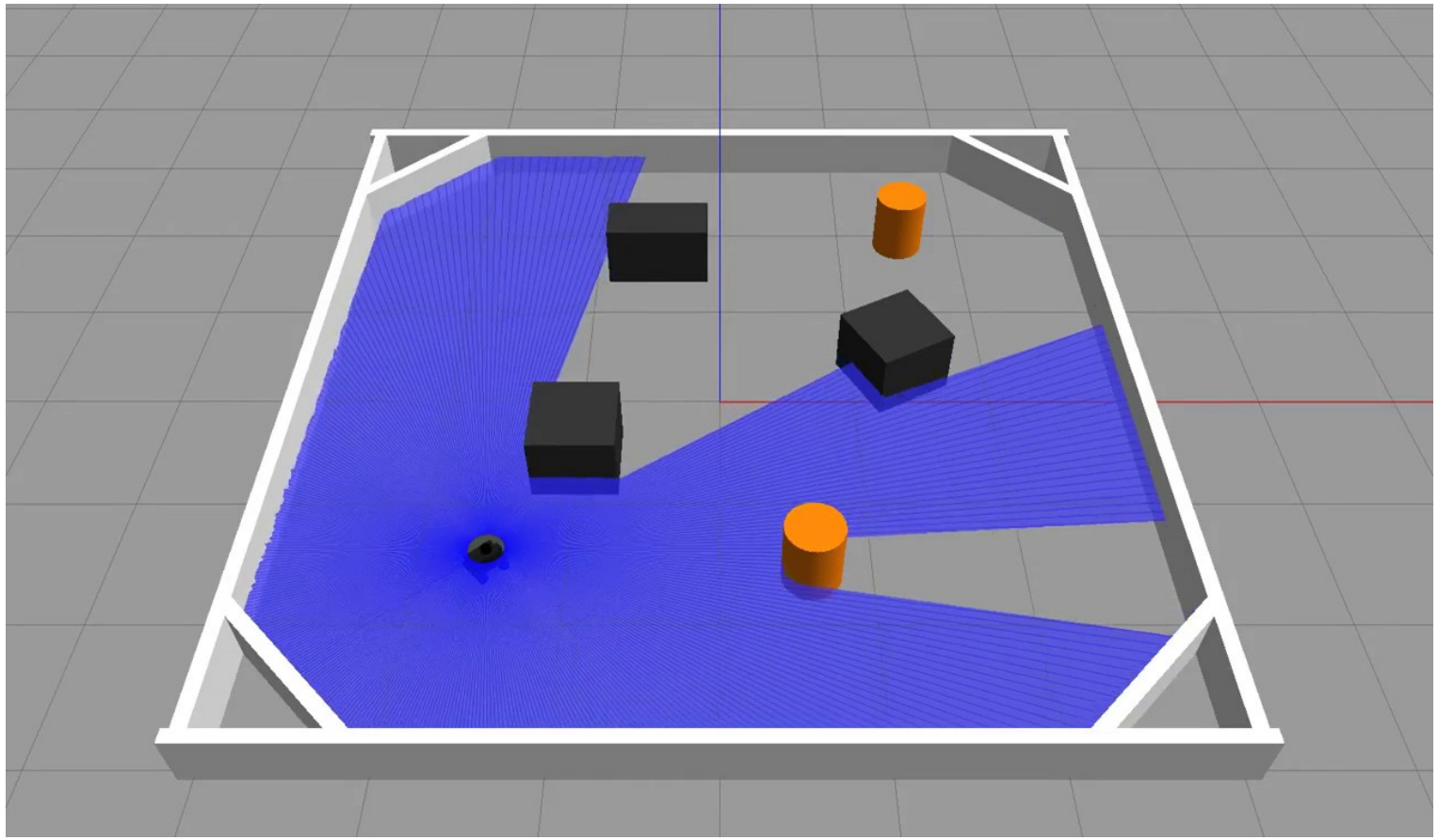
# 4. Terminal 2 (source first): start the node
ros2 run rosmaster_avoidance lidar_avoidance
```

Step 4: Start the node

Our node reads /scan and sends /cmd_vel. The robot starts driving and the terminal prints what it decides.

```
FORWARD  front=2.31 left=1.05 right=0.88
```

EXPECTED OUTPUT



Gazebo with the LiDAR rays shown in blue

Blue rays = /scan

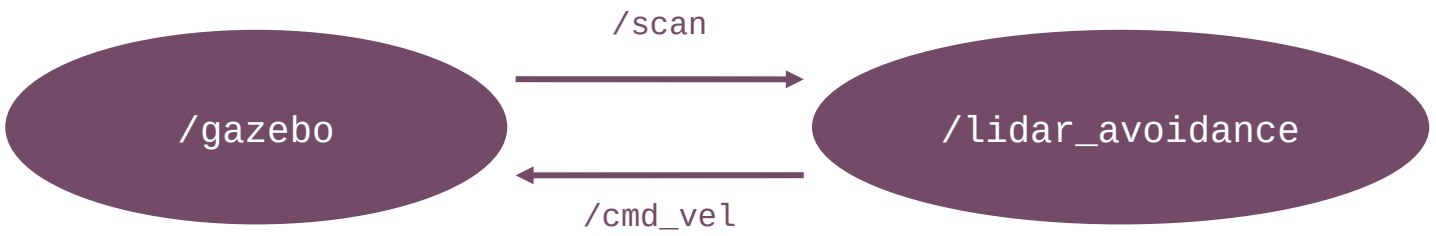
- Each ray is one LiDAR distance
- A ray stops where it hits something
- Behind obstacles there are no rays: the robot can't see there
- The robot turns to the side with longer rays

CHECKING WITH ROS2 TOOLS

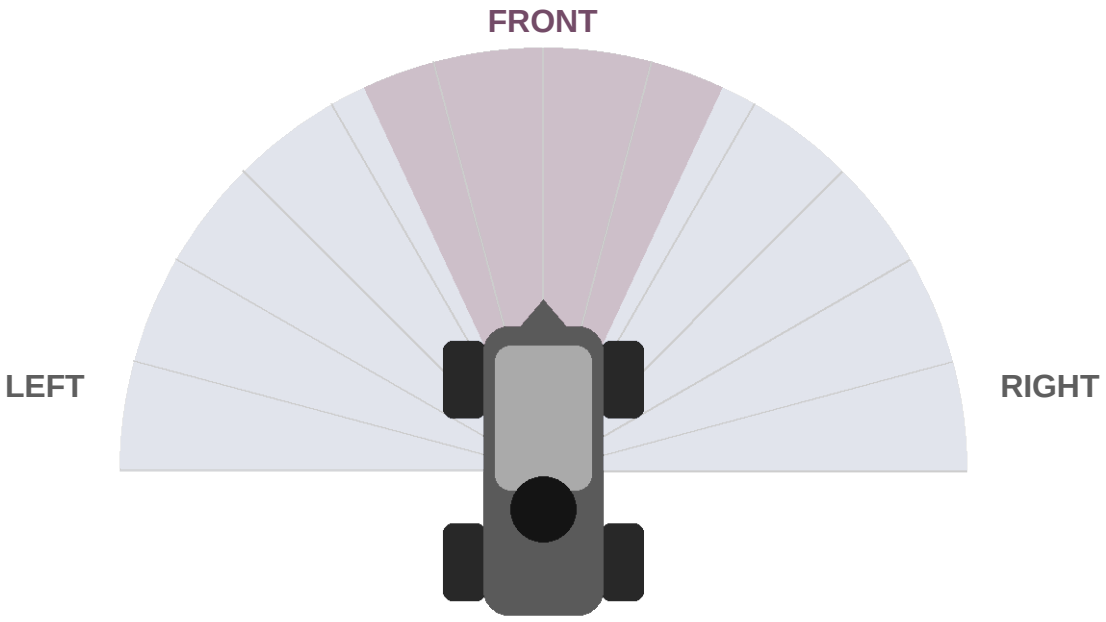
```
# terminal 3 (source first)
ros2 node list             # lidar_avoidance?
ros2 topic list            # /scan and /cmd_vel?
ros2 topic echo /cmd_vel   # commands from our node
rqt_graph
```

github.com/Phearamony/
ROSMASTER_crash_course

rqt_graph should look like this:



HOW THE ROBOT SEES



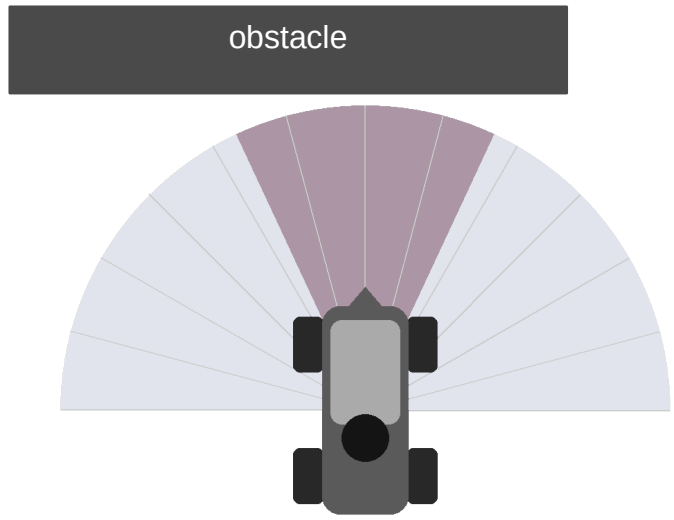
The LiDAR measures the distance to the closest thing in every direction: 360 numbers, about 10 times per second. That is the /scan message.

Our node only keeps the closest distance in 3 zones:

Zone	Angles
`front`	-25° to 25°
`left`	25° to 90°
`right`	-90° to -25°

0° = straight ahead, + = left

AVOIDANCE RULES



1 **Front is clear**

drive forward

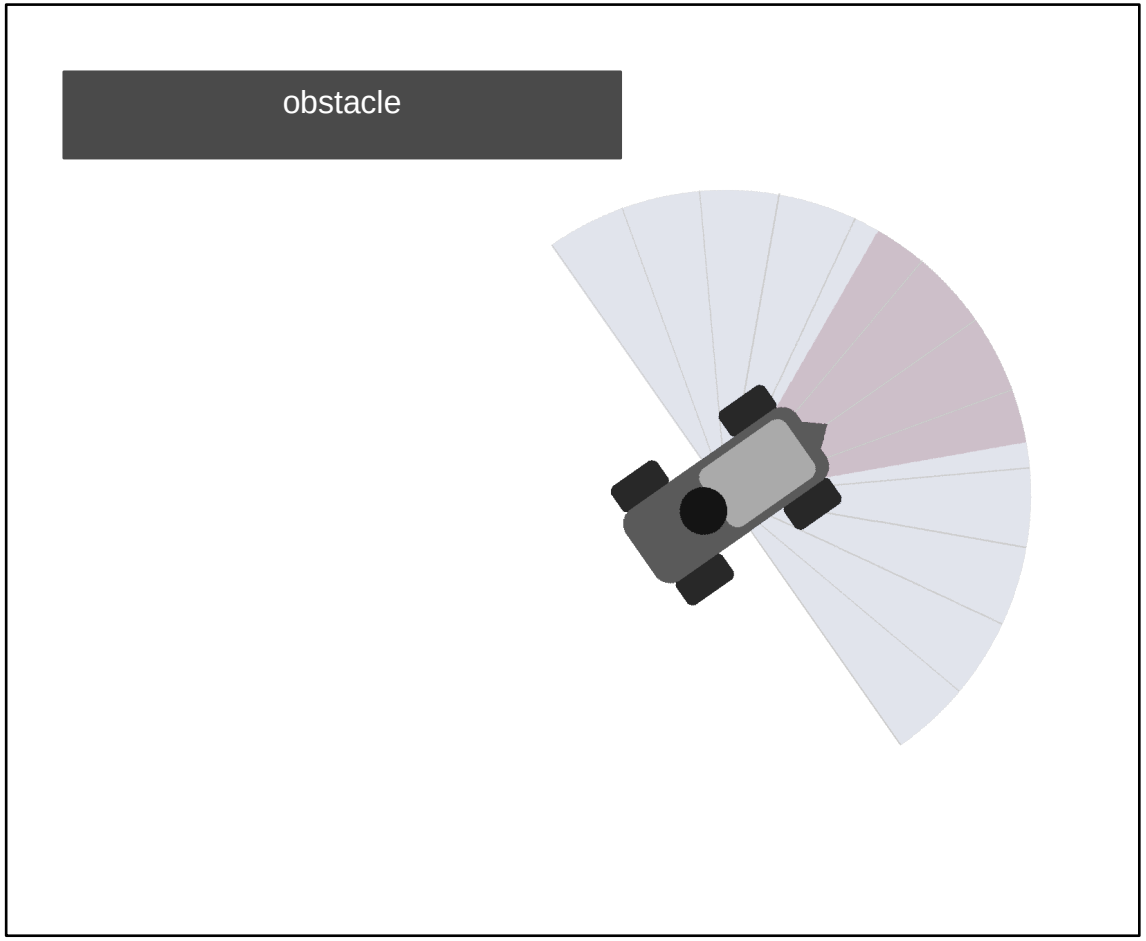
2 **Front is blocked**

slow down, steer to the side with more space

3 **Too close**

back up for a moment

AVOIDANCE RULES



1 **Front is clear**

drive forward

2 **Front is blocked**

slow down, steer to the side with more space

3 **Too close**

back up for a moment

THE CODE: LIDAR_AVOIDANCE.PY

```
class LidarAvoidance(Node):
    def __init__(self):
        super().__init__('lidar_avoidance')
        self.declare_parameter('forward_speed', 0.25)
        self.declare_parameter('safe_distance', 0.7)
        ...
        self.scan_sub = self.create_subscription(
            LaserScan, '/scan', self.scan_callback, 10)
        self.cmd_pub = self.create_publisher(
            Twist, '/cmd_vel', 10)

    def scan_callback(self, scan):
        front = self.sector_min(scan, -25, 25)
        left = self.sector_min(scan, 25, 90)
        right = self.sector_min(scan, -90, -25)

        cmd = Twist()
        if front < stop:
            cmd.linear.x = -0.15
        elif front < safe:
            cmd.linear.x = forward_speed * 0.6
            cmd.angular.z = turn_speed if left > right else -turn_speed
        else:
            cmd.linear.x = forward_speed
        self.cmd_pub.publish(cmd)
```

Subscriber and publisher

Subscribe to /scan, publish to /cmd_vel. Every time a scan comes in, ROS calls scan_callback.

1 / 4

THE CODE: LIDAR_AVOIDANCE.PY

```
class LidarAvoidance(Node):
    def __init__(self):
        super().__init__('lidar_avoidance')
        self.declare_parameter('forward_speed', 0.25)
        self.declare_parameter('safe_distance', 0.7)
        ...
        self.scan_sub = self.create_subscription(
            LaserScan, '/scan', self.scan_callback, 10)
        self.cmd_pub = self.create_publisher(
            Twist, '/cmd_vel', 10)

    def scan_callback(self, scan):
        front = self.sector_min(scan, -25, 25)
        left = self.sector_min(scan, 25, 90)
        right = self.sector_min(scan, -90, -25)

        cmd = Twist()
        if front < stop:
            cmd.linear.x = -0.15
        elif front < safe:
            cmd.linear.x = forward_speed * 0.6
            cmd.angular.z = turn_speed if left > right else -turn_speed
        else:
            cmd.linear.x = forward_speed
        self.cmd_pub.publish(cmd)
```

Parameters

Settings with a default value. You can change them from the terminal without editing the code.

THE CODE: LIDAR_AVOIDANCE.PY

```
class LidarAvoidance(Node):
    def __init__(self):
        super().__init__('lidar_avoidance')
        self.declare_parameter('forward_speed', 0.25)
        self.declare_parameter('safe_distance', 0.7)
        ...
        self.scan_sub = self.create_subscription(
            LaserScan, '/scan', self.scan_callback, 10)
        self.cmd_pub = self.create_publisher(
            Twist, '/cmd_vel', 10)

    def scan_callback(self, scan):
        front = self.sector_min(scan, -25, 25)
        left = self.sector_min(scan, 25, 90)
        right = self.sector_min(scan, -90, -25)

        cmd = Twist()
        if front < stop:
            cmd.linear.x = -0.15
        elif front < safe:
            cmd.linear.x = forward_speed * 0.6
            cmd.angular.z = turn_speed if left > right else -turn_speed
        else:
            cmd.linear.x = forward_speed
        self.cmd_pub.publish(cmd)
```

Read the zones

sector_min goes through the 360 distances and returns the closest one in each zone.

THE CODE: LIDAR_AVOIDANCE.PY

```
class LidarAvoidance(Node):
    def __init__(self):
        super().__init__('lidar_avoidance')
        self.declare_parameter('forward_speed', 0.25)
        self.declare_parameter('safe_distance', 0.7)
        ...
        self.scan_sub = self.create_subscription(
            LaserScan, '/scan', self.scan_callback, 10)
        self.cmd_pub = self.create_publisher(
            Twist, '/cmd_vel', 10)

    def scan_callback(self, scan):
        front = self.sector_min(scan, -25, 25)
        left = self.sector_min(scan, 25, 90)
        right = self.sector_min(scan, -90, -25)

        cmd = Twist()
        if front < stop:
            cmd.linear.x = -0.15
        elif front < safe:
            cmd.linear.x = forward_speed * 0.6
            cmd.angular.z = turn_speed if left > right else -turn_speed
        else:
            cmd.linear.x = forward_speed
        self.cmd_pub.publish(cmd)
```

Decide and send

Twist is the command. linear.x = forward speed (m/s), angular.z = turning (+ = left). Then publish it.

4 / 4

TRY IT

1 Drive it yourself

Stop the node (Ctrl+C), then drive with i, j, l, comma. k stops.

```
ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

2 Look at the data

Drive next to a box, find the small numbers in ranges.

```
ros2 topic echo /scan --once
```

3 Change the behavior

Try 0.4 and 1.0. Which one hits things? Which one gets stuck?

```
ros2 run rosmaster_avoidance lidar_avoidance
  --ros-args -p safe_distance:=1.0
```

4 Add obstacles

Use the box and cylinder buttons in the Gazebo toolbar. Can it still get out?

NEXT WEEK: READING THE CODE

```
def scan_callback(self, scan):
    front = self.sector_min(scan, -25, 25)
    left = self.sector_min(scan, 25, 90)
    right = self.sector_min(scan, -90, -25)

    cmd = Twist()
    if front < safe:
        cmd.linear.x = forward_speed * 0.6
        cmd.angular.z = turn_speed if left > right else -turn_speed
```

Python

variables, if, functions, classes

Bash

cd, ls, nano, source, sudo apt

Linux

files, folders, permissions, terminal

Homework: do the 4 experiments, bring 1 question.